

Fitting the page into a prompt

Week 3, Lecture 2 · Browser Use: How LLM Agents Drive the Web

Summer 2026 · Capturing browser state: the session layer

What we'll cover

- Why raw HTML cannot fit a model context window
- What the context budget is and what competes for it
- Ephemeral messages: which parts of the context expire and which do not
- The last-N browser-state retention strategy
- Token cost of vision vs DOM: when each is worth it
- How the agent's memory field compensates for dropped history

Why raw HTML does not work

The size problem

A medium-complexity web application page in raw HTML:

- Inline JavaScript: often 10,000-50,000 characters
- Inline CSS and SVG: another 5,000-30,000 characters
- Nested `div` containers, data attributes, class names: structural noise
- Total: commonly 50,000-300,000 characters = 12,000-75,000 tokens

A frontier model context window is 32,000-200,000 tokens. The raw HTML of one page can consume a large portion of it before the system prompt, task, or history are included.

(Gonsalvez, 2025)

The signal-to-noise problem

Even if the HTML fit, most of it is not useful for navigation:

```
<div class="hn-main" data-v-3b2c9a1d="">
  <table border="0" cellpadding="0" cellspacing="0" class="fatitem">
    <tbody>
      <tr class="athing submission" id="42052432">
        <td class="title" valign="top">
          <span class="titleline">
            <a href="https://github.com/browser-use/browser-use">
              Show HN: Open-source browser use
            </a>
```

What the agent needs: "Link [3]: Show HN: Open-source browser use".

The DOM service extracts exactly that, discarding the rest.

The filtered DOM: what actually goes to the LLM

After filtering and indexing, the same content looks like:

```
[3] Link: "Show HN: Open-source browser use" → github.com/browser-use/browser-use  
[4] Button: "vote" (upvote story 42052432)  
[5] Link: "143 comments" → item?id=42052432
```

Three lines instead of twelve. All the action-relevant content, none of the markup noise.

A typical Hacker News front page: from ~80,000 characters of raw HTML to ~3,000 characters in the indexed element tree.

The context budget

What competes for context space

Each agent step assembles a message list for the LLM. The budget has to cover:

Component	Ephemeral?	Typical size
System prompt + task	No	500-2,000 tokens
Current browser-state summary	Yes (current)	500-3,000 tokens
Recent browser-state summaries	Yes (dropped over time)	500-3,000 per step
Action history and results	Partially	100-500 per step
Agent memory field	No	100-1,000 tokens
LLM output (reserved)	(reserved space)	500-2,000 tokens

The context window fills from the top. Older content competes with newer content for the remaining space.

How the budget shrinks step by step

At step 1: system prompt + task + state 1 = small

At step 5: system prompt + task + states 1-5 + history = larger

At step 15: system prompt + task + states 1-15 + history = potentially too large

Without management, the context window fills and the run fails or degrades. The message manager handles this by making some content ephemeral.

(browser-use contributors, 2026)

Ephemeral messages

What "ephemeral" means

An ephemeral message is one that is included in the context for a limited number of steps, then removed to free budget.

Ephemeral: browser-state summaries from earlier steps. Step 1's state is in the context at step 2, but may be gone by step 10.

Non-ephemeral (persistent):

- The system prompt
- The task description
- The agent's `memory` field

The task never expires. The memory does not expire. Old page states do expire.

The last-N retention strategy

The message manager keeps the most recent N browser-state summaries and drops older ones. The exact value of N depends on the model context size and configuration.

Why this is correct for web navigation:

- The current page state is always relevant
- A page from 2-3 steps ago may be relevant (browser back, multi-page form)
- A page from 15 steps ago (visited and left) is almost never needed again
- If old context is needed, the agent can navigate back

The sliding window moves forward with each step.

The memory field as compensation

When a browser-state summary is dropped, any facts from that page are lost unless the agent recorded them.

The `memory` field in the agent's structured output (covered in week 4) is a persistent note the agent carries forward across steps:

```
memory: "User is logged in as alice@example.com. Search returned  
42 results. Product ID for blue widget is SKU-88241."
```

This survives the ephemeral drop. The agent's strategy: when a fact from the current state will be needed later, write it to memory now.

(browser-use contributors, 2026)

Vision vs DOM: token cost

DOM-only token cost

A typical state summary without a screenshot:

- URL + title: ~20 tokens
- Element tree (30-80 elements with text): 500-2,000 tokens
- Selector map (keys only, values stored internally): minimal
- **Total per step: ~500-2,000 tokens**

This is the default. Any capable text model can use it.

Vision: what it adds

With `use_vision=True`, a base64-encoded PNG of the viewport is added to the state:

- A 1280x800 screenshot at medium quality: ~60-120 KB
- Base64 encoding adds ~33% size: ~80-160 KB of string
- At low-detail image token pricing: ~500-1,000 additional tokens
- At high-detail image token pricing: ~1,500-3,000 additional tokens

For a 30-step run at ~750 tokens per screenshot: 22,500 additional tokens beyond DOM cost.
At \$0.003/1K input tokens (rough example, not a current price): approximately \$0.07 extra for screenshots alone.

Verify current pricing at your provider before estimating real run cost.

When vision helps

Vision adds information the text DOM cannot carry:

- **Visual layout matters:** calendar grids, spatial positioning of options, image carousels where the order or image content is the task
- **Canvas elements:** charts, custom drawing areas. These have no DOM nodes, only a `<canvas>` tag.
- **Visual confirmation:** the agent can see whether a button is highlighted, a checkbox is checked, or a modal appeared. Useful when CSS styling carries semantic meaning the DOM text does not.

For standard navigation, form filling, and text extraction: DOM-only is accurate and cheaper. (Gonsalvez, 2025)

The HTML of a typical modern web app is not a clean semantic document. It's a delivery artifact. What the model needs is the semantic summary underneath it.

Steven Gonsalvez (2025)

Take-aways

1. Raw HTML commonly runs 12,000-75,000 tokens per page. The filtered DOM is an order of magnitude smaller and contains only action-relevant content.
2. The context budget covers the system prompt, task, current and recent browser states, action history, and the agent memory. All compete for the same window.
3. Ephemeral messages are browser-state summaries that are dropped as the run progresses. Non-ephemeral content (task, memory) persists.
4. The last-N retention strategy keeps recent states and drops old ones. The agent's memory field compensates for lost history.
5. Vision adds ~500-3,000 tokens per step. Use it when visual layout or canvas content is essential; skip it for text-based navigation tasks.

What's next

- **Section this week:** Configure two sessions and compare their state summaries (see the token cost difference directly)
- **HW2:** Trace the state-to-action pipeline (assigned this week)
- **Week 4:** The decision step: how the LLM turns the state summary into a structured action list (AgentOutput and function calling)